



GUÍA TÉCNICA UTP

Despliegue de Spring Boot en Render y conexión con MySQL

Enfoque práctico: GitHub + Docker + Render + Base de datos

RESUMEN

Recomendación docente: primero lograr que el servicio Spring Boot levante en Render con Docker; luego conectar una base de datos persistente. Para Render, PostgreSQL administrado es la opción más simple. MySQL se puede usar, pero normalmente requiere un servicio privado con Docker y disco persistente.

Preparado para clase y laboratorio técnico



Contenido de la guía

- Decisión técnica: MySQL en Render vs PostgreSQL administrado.
- Arquitectura recomendada para Spring Boot desplegado desde GitHub.
- Archivos mínimos: Dockerfile, application.properties, pom.xml y variables de entorno.
- Despliegue inicial del backend sin base real usando H2 temporal.
- Opción A: conexión recomendada a Render PostgreSQL.
- Opción B: despliegue de MySQL en Render como servicio privado con Docker y disco persistente.
- Migración desde H2 temporal hacia MySQL.
- Pruebas, troubleshooting y checklist final.

SUPUESTO

Esta guía asume que el proyecto Spring Boot ya está subido a GitHub en la rama main y que Render está configurado con Language = Docker.

1. ¿Se puede desplegar MySQL en Render?

Sí, se puede desplegar MySQL en Render, pero no como una base de datos administrada igual que PostgreSQL. Render ofrece PostgreSQL administrado y también permite ejecutar bases personalizadas como MySQL usando Docker y un disco persistente.

La decisión técnica depende de tu objetivo: si quieres rapidez, menor mantenimiento y plan gratuito temporal, conviene PostgreSQL. Si tu proyecto o curso exige MySQL, entonces puedes desplegar MySQL como Private Service con Docker, pero debes cuidar el disco persistente, variables de entorno y backups.

Opción	Uso recomendado	Ventaja	Cuidado técnico
PostgreSQL en Render	Proyectos académicos y despliegue rápido	Es administrado y se crea desde New > Postgres	El plan free tiene limitaciones y puede expirar
MySQL en Render	Cuando el proyecto exige MySQL	Compatible con proyectos Spring Boot ya hechos en MySQL	Requiere Docker, Private Service y disco persistente
MySQL externo	Cuando se desea menor mantenimiento dentro de Render	Puede usarse Railway, PlanetScale, Aiven, Clever Cloud u otro proveedor	La conexión será externa y puede haber latencia/costos
H2 temporal	Solo para probar que el backend levanta	Muy rápido para laboratorio	Los datos no son persistentes

REGLA

Para producción real, no confíes en H2 ni en archivos locales. Usa una base persistente y define backups.

2. Arquitectura objetivo

La arquitectura recomendada separa el backend Spring Boot y la base de datos. El backend se despliega como Web Service en Render desde GitHub usando Docker. La base de datos puede ser PostgreSQL administrado o MySQL como servicio privado.

Diagrama lógico

```

Usuario / Frontend
  |
  v
https://tu-api.onrender.com
  |
  v
Spring Boot Web Service en Render
  |
  | Variables de entorno
  | DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD
  v
Base de datos persistente
- Opción A: Render PostgreSQL administrado
- Opción B: MySQL Private Service + Disk /var/lib/mysql

```

SEGURIDAD

El backend nunca debe tener usuario y contraseña escritos directamente en el código. Usa variables de entorno en Render.

3. Despliegue inicial de Spring Boot en Render

Paso	Acción técnica	Resultado esperado
1	Verificar que GitHub tenga el proyecto descomprimido, no solo un archivo ZIP.	Render puede leer pom.xml, src/ y Dockerfile.
2	Seleccionar Language = Docker en Render.	Render compila la imagen desde el Dockerfile.
3	Usar Branch = main.	Render toma el último commit de la rama principal.
4	Dejar Root Directory vacío si pom.xml y Dockerfile están en la raíz.	Evita errores de ruta.
5	Configurar server.port=\${PORT:8080}.	La app escucha el puerto asignado por Render.
6	Revisar logs hasta ver Started ...Application.	Confirma que el backend levantó correctamente.

Estructura mínima del repositorio

```

modelo_desarrolloweb_int/
├── src/
├── pom.xml
├── Dockerfile
├── mvnw
├── mvnw.cmd
└── .gitignore

```

4. Dockerfile recomendado para Spring Boot

Este Dockerfile compila el proyecto con Maven y luego ejecuta el JAR generado en una imagen liviana de Java Runtime. Debe colocarse en la raíz del repositorio.

Dockerfile

```

FROM maven:3.9.9-eclipse-temurin-17 AS build

WORKDIR /app

COPY pom.xml .
COPY src ./src

RUN mvn clean package -DskipTests

FROM eclipse-temurin:17-jre

WORKDIR /app

COPY --from=build /app/target/*.jar app.jar

EXPOSE 10000

ENTRYPOINT ["java", "-jar", "app.jar"]

```

AJUSTE

Java 21: cambia 17 por 21 en ambas imágenes.

5. Configuración temporal con H2 para probar el servicio

Antes de conectar una base real, puedes levantar la API usando H2 en memoria. Esto sirve para validar que Render, Docker y Spring Boot están funcionando.

Dependencia temporal en pom.xml

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
```

application.properties temporal

```
server.port=${PORT:8080}

spring.datasource.url=jdbc:h2:mem:renderdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.h2.console.enabled=true
```

ADVERTENCIA

H2 solo es temporal. Cada reinicio puede borrar los datos. Cuando el backend esté live, cambia a una base persistente.

6. Opción recomendada: usar Render PostgreSQL

Aunque tu proyecto haya nacido con MySQL, para Render la opción más simple y administrada es PostgreSQL. Render permite crear una base desde New > Postgres y entrega una Internal URL para conectar el backend en la misma región.

Paso	Acción técnica	Resultado esperado
1	En Render, ir a New > Postgres.	Se abre el formulario de creación de base de datos.
2	Colocar nombre: modelo-desarrolloweb-db.	Nombre claro para reconocerla.
3	Seleccionar la misma región del Web Service.	Menor latencia y conexión interna.
4	Crear la base y esperar estado Available.	La base ya puede recibir conexiones.
5	Copiar Internal Database URL o credenciales separadas.	Se usarán como variables de entorno.
6	Agregar variables en el Web Service Spring Boot.	El backend queda conectado a la base persistente.

Dependencia PostgreSQL en pom.xml

```
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
```

application.properties para PostgreSQL

```
server.port=${PORT:8080}

spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASSWORD}

spring.jpa.hibernate.ddl-auto=update
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.show-sql=true
```

Variables de entorno en Render

```
DB_URL=jdbc:postgresql://HOST_INTERNO:5432/NOMBRE_DB
DB_USER=USUARIO_DB
DB_PASSWORD=PASSWORD_DB
```

COMPATIBILIDAD

Si usas PostgreSQL, revisa nombres de tablas y palabras reservadas. En proyectos simples con JPA suele funcionar con pocos cambios.

7. Opción solicitada: desplegar MySQL en Render

MySQL en Render se trabaja como servicio privado con Docker. La idea es que el contenedor MySQL no esté expuesto públicamente; el backend Spring Boot se comunica con él por la red privada de Render usando el host interno.

Paso	Acción técnica	Resultado esperado
1	Crear o usar el template oficial render-examples/mysql.	Se usa la imagen oficial de MySQL.
2	Crear New > Private Service en Render.	MySQL queda accesible solo desde servicios del workspace.
3	Seleccionar Environment/Language = Docker.	Render ejecuta MySQL en contenedor.
4	Configurar variables MYSQL_DATABASE, MYSQL_USER, MYSQL_PASSWORD y MYSQL_ROOT_PASSWORD.	MySQL inicia con base y usuario.
5	Agregar Persistent Disk con Mount Path /var/lib/mysql.	Los datos sobreviven a reinicios y reddeploys.
6	Esperar que el servicio MySQL quede Live.	Render mostrará una URL privada tipo mysql-foo:3306.
7	Agregar credenciales de MySQL al Web Service Spring Boot.	La API se conecta al host privado de MySQL.

CRÍTICO

El Mount Path para MySQL debe ser exactamente /var/lib/mysql. Si colocas otra ruta, MySQL seguirá escribiendo en su ruta interna y puedes perder datos.

7.1 Variables típicas para el servicio MySQL

Variables del servicio MySQL

```
MYSQL_DATABASE=modelo_db
MYSQL_USER=modelo_user
MYSQL_PASSWORD=una_clave_segura
MYSQL_ROOT_PASSWORD=otra_clave_muy_segura
```

En Render, estas variables se configuran dentro del servicio MySQL, no dentro del backend. Luego, en el backend, agregas las variables de conexión hacia ese MySQL.

7.2 Configuración del backend Spring Boot para MySQL

Dependencia MySQL en pom.xml

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>
```

application.properties para MySQL

```
server.port=${PORT:8080}

spring.datasource.url=jdbc:mysql://${DB_HOST}:${DB_PORT}/${DB_NAME}?
useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
spring.datasource.username=${DB_USER}
spring.datasource.password=${DB_PASSWORD}

spring.jpa.hibernate.ddl-auto=update
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
spring.jpa.show-sql=true
```

Variables de entorno del Web Service Spring Boot

```
DB_HOST=mysql-xxxx
DB_PORT=3306
DB_NAME=modelo_db
DB_USER=modelo_user
DB_PASSWORD=una_clave_segura
```

ERROR COMÚN

DB_HOST no debe ser localhost. En Render debe ser el hostname privado que muestra el servicio MySQL, por ejemplo mysql-foo o mysql-foo:3306 separado en host y puerto.

8. Migración desde H2 temporal hacia MySQL

Paso	Acción técnica	Resultado esperado
1	Quitar o dejar sin uso la configuración H2 en application.properties.	Spring Boot deja de usar la base en memoria.
2	Confirmar que pom.xml tenga mysql-connector-j.	El driver MySQL estará disponible.
3	Cambiar spring.datasource.url a variables DB_HOST, DB_PORT y DB_NAME.	No se exponen credenciales en GitHub.
4	Agregar variables DB_* en Render Web Service.	Render inyecta las credenciales al iniciar.
5	Ejecutar Manual Deploy > Deploy latest commit.	Se recompila la app con la configuración final.
6	Probar endpoints GET/POST.	Las operaciones deben persistir en MySQL.

Commit recomendado

```
git add .
git commit -m "Configurar conexion MySQL para Render"
git push
```

9. Pruebas técnicas posteriores al deploy

Una vez que Render muestre Deploy Live, prueba los endpoints principales. Si tu API tiene Swagger, valida también la documentación interactiva.

URLs de prueba

```
GET https://tu-api.onrender.com/
GET https://tu-api.onrender.com/productos
POST https://tu-api.onrender.com/productos
DELETE https://tu-api.onrender.com/productos/{id}
GET https://tu-api.onrender.com/swagger-ui/index.html
```

- Si GET responde, el servicio web está levantado.
- Si POST guarda y luego GET lista el nuevo registro, la base de datos está conectada.
- Si al redeployar los datos siguen existiendo, la persistencia está funcionando.
- Si aparece 502, revisar puerto y logs de arranque.
- Si aparece Communications link failure, revisar host, puerto, usuario y contraseña de MySQL.

10. Troubleshooting rápido

Problema	Causa probable	Solución
502 Bad Gateway	La app no escucha el puerto correcto	Usar server.port=\${PORT:8080} y revisar logs.
No open ports detected	Spring Boot no inició o escucha otro puerto	Revisar Dockerfile, JAR y application.properties.
Build failed	Dockerfile, pom.xml o Java version incorrecta	Verificar que Dockerfile y pom.xml estén en la raíz.
Access denied for user	Usuario o contraseña incorrectos	Corregir DB_USER y DB_PASSWORD en Environment.
Unknown database	DB_NAME no existe	Crear la base o verificar MYSQL_DATABASE.
Communications link failure	Host/puerto incorrecto o DB no está Live	Usar host privado de Render y puerto 3306.
Datos perdidos	No hay disco persistente o mount path incorrecto	En MySQL usar Disk con /var/lib/mysql.

11. Checklist final para clase

- Repositorio GitHub con src/, pom.xml y Dockerfile visibles.
- Render Web Service creado con Language = Docker y Branch = main.
- application.properties usando server.port=\${PORT:8080}.
- Servicio Spring Boot en estado Deploy Live.
- Endpoint base probado en navegador o Postman.
- Base de datos definida: H2 temporal, PostgreSQL o MySQL.
- Credenciales definidas como variables de entorno, nunca escritas en GitHub.
- Para MySQL: servicio privado Live, puerto 3306 y disco en /var/lib/mysql.
- Para persistencia: probar POST, redeploy y GET nuevamente.
- Logs revisados y documentados para evidencia de despliegue.

12. Recomendación final

Para una clase práctica, el mejor orden es: primero desplegar Spring Boot con H2 temporal; segundo validar endpoints; tercero conectar PostgreSQL si quieres rapidez; cuarto usar MySQL si el curso exige específicamente MySQL. Para MySQL en Render, considera que se administra como contenedor con disco persistente, por lo que requiere más responsabilidad técnica que PostgreSQL administrado.

CIERRE

Mi recomendación técnica: para estudiantes, usar PostgreSQL en Render si el objetivo es aprender despliegue cloud. Usar MySQL en Render si el objetivo es aprender Docker, red privada, persistencia y administración básica de servicios.

Fuentes oficiales consultadas

- Render Docs - Docker on Render: <https://render.com/docs/docker>
- Render Docs - Deploy MySQL: <https://render.com/docs/deploy-mysql>
- Render Docs - Persistent Disks: <https://render.com/docs/disks>
- Render Docs - Create and Connect to Render Postgres: <https://render.com/docs/postgresql-creating-connecting>
- Render Docs - Deploy for Free: <https://render.com/docs/free>
- GitHub render-examples/mysql: <https://github.com/render-examples/mysql>